SmartTable

Codebase Documentation

Generated from the current repository

Generated date: 2026-07-18

Secrets, passwords, tokens, and production credentials are intentionally omitted.

Scope

Working, partial, demo, disabled, and planned features are separated.

No admin-interface changes are made by this documentation generator.

Table of Contents

Entries are clickable in PDF viewers that support internal document links.

Project Overview

SmartTable is a discounted restaurant reservation marketplace with a static browser UI, a Node API layer, Supabase-ready PostgreSQL migrations, demo-mode fallback data, Resend-ready transactional email, partner/admin dashboards, and a gated AI Concierge mode.

SmartTable integrates with reservation systems only. It does not connect to restaurant POS systems or access restaurant payment and transaction data.

Current local platform settings: platform_mode=basic, ai_demo_visibility=False, show_ai_mode_badge=True.

Runtime item	Location
Static frontend	public/app.js, public/index.html, public/styles.css
Local server	server.js
API core	src/app-core.js
Vercel API	api/index.js, api/[...path].js
Provider abstraction	src/reservation-providers.js
Database migrations	supabase/migrations/*.sql

Feature Status Summary

The current codebase intentionally separates working marketplace features from beta, demo, disabled, and integration-dependent AI modules.

Status	Meaning	Examples
Working	Frontend, API, and data flow exist.	Public offers, reservation requests, admin/partner basics, Platform Mode.
Partial / Beta	Structures and UI/API exist but need hardening or production workflows.	AI recommendations, imports, monitoring, reviews, billing foundation.
Demo only	Mock or deterministic behavior only.	Partner AI mock analytics, demo AI Concierge cards.
Disabled	Registered but unavailable.	Calendar sync.
Planned / Requires integration	Schema/placeholders exist; provider access is not connected.	OpenTable, Resy, Stripe, OpenAI, weather/events.

Basic and AI Concierge Modes

Platform Mode is stored in local demo settings or Supabase app_settings. Default mode is basic. Super Admin can switch it through the admin platform settings API and UI.

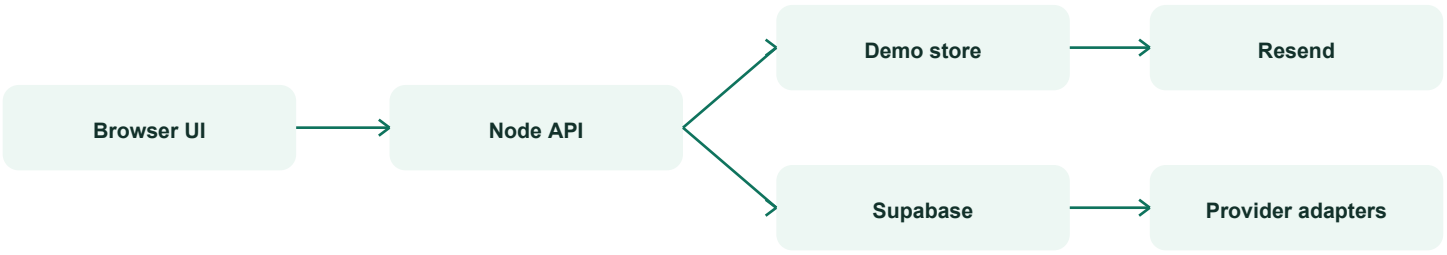
Mode	Behavior
basic	Shows the non-AI discounted reservation marketplace and hides AI navigation/claims.
ai_concierge	Allows AI navigation and AI sections when feature registry rules and demo visibility allow them.

AI Demo Visibility is separate. Demo features remain hidden unless AI_CONCIERGE mode and ai_demo_visibility are both enabled.

Guest, Partner, Admin, Super Admin Functions

Audience	Implemented functions
Guest	Browse offers, open restaurant detail/modal, request reservations, follow restaurants, submit reviews/feedback/photos, use EN/ES/HU, access visible AI Concierge when enabled.
Partner	Login, edit profile, manage offers, view/update reservations, view stats, integrations/imports, photo feedback, AI Demand entry when enabled.
Admin	Manage restaurants, partners, offers, reservations, content, notifications, reviews, integrations, feature flags, billing foundation, monitoring, privacy requests, stats.
Super Admin	All admin functions plus Platform Mode switching and partner impersonation/view-as flow.

Architecture



- server.js serves static assets and delegates /api/* requests to handleApiRequest.
- api/index.js provides the Vercel serverless API entry point.
- src/app-core.js contains business logic, Supabase access, permissions, emails, demo fallback data, Platform Mode, and feature registry.
- src/reservation-providers.js defines generic and mock OpenTable/Resy/SevenRooms/Tock provider adapters.
- public/app.js contains the browser single-page app and feature-visibility logic.

Folder Structure

Path	Purpose
api/	Vercel API entry files.
data/app-settings.json	Local demo persistence for Platform Mode settings.
public/	Frontend, styles, assets, language files, manifest, robots, sitemap.
scripts/	Checks, locale utilities, documentation generator.
src/	API core and integration/provider abstraction.
supabase/migrations/	PostgreSQL schema, views, RLS, seeds, settings.
backups/	Manual/autosave snapshots, not runtime source.

Frontend Route Structure and Redirects

The browser app is a single-page application with shared backend, shared auth, shared API, shared database, shared translations, and shared platform settings.

Area	Routes
Guest public	/, /restaurants, /restaurants/:slug, /offers, /signup, /login, /forgot-password, /reset-password, /terms, /privacy, /contact, /help
Guest protected	/account, /account/reservations, /account/favorites, /account/profile, /account/preferences, /account/notifications, /account/reviews, /account/security
Partner	/partner, /partner/offers, /partner/reservations, /partner/profile, /partner/analytics, /partner/settings, /partner/ai-demand when allowed
Admin	/admin, /admin/restaurants, /admin/offers, /admin/users, /admin/notifications, /admin/content, /admin/platform-settings, /admin/ai-controls when allowed
Compatibility	Direct URL refresh uses SPA fallback. Hash aliases such as #guest-signup, #partner-ai-demand, and #admin-ai-controls remain supported.

SEO, Mobile, and Security Cleanup

- Route-aware title, meta description, canonical, robots, and Open Graph updates exist in server.js and public/app.js.
- robots.txt and sitemap.xml are served dynamically by server.js and have static fallbacks in public/.
- Private admin, partner, account, login/reset, and post-visit upload routes are noindex.
- Mobile cleanup covers narrow phones, modal containment, signup option grids, filters, account cards, and 44px touch targets.
- Public offers are sanitized to avoid leaking restaurant email, owner IDs, partner/admin notes, roles, permissions, tokens, secrets, or private keys.
- Security headers include nosniff, DENY frame policy, strict-origin referrer policy, and restricted permissions policy.

Reservation Integration Boundaries and POS Ban

SmartTable integrates with reservation systems only. It does not connect to restaurant POS systems or access restaurant payment and transaction data.

Allowed	Not allowed
Resy, OpenTable, SevenRooms, Tock, Google Reserve, approved reservation APIs, CSV/manual reservation import.	Toast POS, Square POS, Clover, Lightspeed, Oracle MICROS, TouchBistro, payment/card/order/sales/inventory/cash-register/tip/refund/settlement data.
Reservation count, available times, table availability, party size, booking status, capacity, active offers, conversions, searches, clicks, favorites, ratings, feedback, weather/events/traffic when separately integrated.	Individual bills, item-level POS sales, employee sales, payment settlements, and any POS-derived revenue estimate.

Subdomain Configuration

The current app can run as one shared application behind different domains or subdomains while keeping one backend, one database, one auth system, and one set of platform settings.

Host	Route target
smarttable.com	Guest marketplace /
www.smarttable.com	Guest marketplace /
partners.smarttable.com	/partner or a rewrite to /partner
admin.smarttable.com	/admin or a rewrite to /admin

Set PUBLIC_BASE_URL to the primary public guest URL used in emails and canonical links. Partner/admin subdomains must remain protected by server-side auth and role checks.

Database

The repository currently has 34 migrations, 60 unique tables, 6 unique views, 80 indexes, and 81 unique RLS policies.

Table	First migration	Purpose
admin_alerts	0024_integration_hub_billing_monitoring.sql	Admin alert records.
admin_notifications	0008_reviews_notifications_newest.sql	Admin notification center.
ai_action_results	0023_real_ai_operating_system_foundation.sql	Measured AI action results.
ai_actions	0023_real_ai_operating_system_foundation.sql	AI recommendation approval/action records.
ai_demand_forecasts	0009_ai_platform_foundation.sql	Demand forecast storage.
ai_interaction_events	0009_ai_platform_foundation.sql	AI learning/interactions event log.
ai_preference_profiles	0009_ai_platform_foundation.sql	Guest AI preference profiles.
ai_processing_jobs	0010_restaurant_intelligence_expansion.sql	Future async AI job queue records.
ai_recommendations	0023_real_ai_operating_system_foundation.sql	AI recommendation records.
ai_route_plans	0010_restaurant_intelligence_expansion.sql	Route planning estimates.
ai_service_time_observations	0010_restaurant_intelligence_expansion.sql	Service duration observations.
analytics_events	0010_restaurant_intelligence_expansion.sql	Generic analytics events.
app_error_logs	0024_integration_hub_billing_monitoring.sql	Application error and audit-like log storage.
app_settings	0027_platform_mode_settings.sql	Persistent platform settings including Platform Mode.
audit_logs	0010_restaurant_intelligence_expansion.sql	Audit/activity log.
billing_plans	0024_integration_hub_billing_monitoring.sql	Billing plan foundation.
calendar_connections	0009_ai_platform_foundation.sql	Future calendar connection records.
data_import_jobs	0024_integration_hub_billing_monitoring.sql	CSV/manual import jobs.
demand_snapshots	0023_real_ai_operating_system_foundation.sql	Demand score/history snapshots.
dining_consumption_uploads	0010_restaurant_intelligence_expansion.sql	Dining photo/review intelligence submissions.
email_events	0001_initial_schema.sql	Legacy/demo email event logging.
email_logs	0023_real_ai_operating_system_foundation.sql	Transactional/campaign email logs.
email_unsubscribes	0024_integration_hub_billing_monitoring.sql	Unsubscribe records.
feature_flags	0024_integration_hub_billing_monitoring.sql	Admin-managed feature flags.
feature_status	0023_real_ai_operating_system_foundation.sql	Feature status registry in database.
guest_auth_events	0031_guest_account_auth_system.sql	Structured table defined by migrations.
guest_consents	0024_integration_hub_billing_monitoring.sql	Consent records.
guest_feedback	0023_real_ai_operating_system_foundation.sql	Post-visit guest feedback and moderation.
guest_notifications	0019_post_visit_photo_rewards.sql	Structured table defined by migrations.
guest_profiles	0023_real_ai_operating_system_foundation.sql	Extended guest preferences/profile data.

Table	First migration	Purpose
guests	0023_real_ai_operating_system_foundation.sql	Guest identity records.
imported_guests	0023_real_ai_operating_system_foundation.sql	Imported guest records.
imported_reservations	0023_real_ai_operating_system_foundation.sql	Imported reservation history.
integration_connections	0023_real_ai_operating_system_foundation.sql	Restaurant integration connections/status.
integration_error_logs	0024_integration_hub_billing_monitoring.sql	Integration error logs.
integration_sync_runs	0024_integration_hub_billing_monitoring.sql	Integration sync run logs.
integrations	0023_real_ai_operating_system_foundation.sql	Integration provider catalog.
invoices	0024_integration_hub_billing_monitoring.sql	Invoice foundation.
legal_documents	0024_integration_hub_billing_monitoring.sql	Terms/privacy document records.
loyalty_accounts	0010_restaurant_intelligence_expansion.sql	Guest loyalty point/badge records.
manual_performance_uploads	0024_integration_hub_billing_monitoring.sql	Manual weekly performance uploads.
marketing_campaigns	0023_real_ai_operating_system_foundation.sql	Campaign records generated manually or by AI approval.
notification_logs	0023_real_ai_operating_system_foundation.sql	Guest notification logs.
offers	0001_initial_schema.sql	Discounted table availability and offer rules.
payment_events	0024_integration_hub_billing_monitoring.sql	Payment event foundation.
privacy_requests	0024_integration_hub_billing_monitoring.sql	Data/privacy request records.
profiles	0001_initial_schema.sql	User profiles and roles.
push_delivery_logs	0034_scale_readiness_feature_flags_booking.sql	Structured table defined by migrations.
push_subscriptions	0034_scale_readiness_feature_flags_booking.sql	Structured table defined by migrations.
reservation_sources	0023_real_ai_operating_system_foundation.sql	External/manual reservation source catalog.
reservations	0001_initial_schema.sql	SmartTable reservation leads and status tracking.
restaurant_followers	0007_restaurant_order_followers_maps.sql	Email-based restaurant follow/favorite subscriptions.
restaurant_integrations	0009_ai_platform_foundation.sql	Legacy restaurant integration settings.
restaurant_reviews	0008_reviews_notifications_newest.sql	Food/service/ambience reviews and moderation status.
restaurant_users	0023_real_ai_operating_system_foundation.sql	Restaurant team-member/account relationships.
restaurant_view_events	0004_saas_platform_content_partner.sql	Restaurant view tracking.
restaurants	0001_initial_schema.sql	Restaurant profile, location, billing, and operating data.
revenue_snapshots	0023_real_ai_operating_system_foundation.sql	Revenue/value snapshots.
site_content	0004_saas_platform_content_partner.sql	Admin-editable public content keys.
subscriptions	0024_integration_hub_billing_monitoring.sql	Restaurant subscription foundation.

Key views include public_available_offers, public_restaurant_cards, reservation_overview, restaurant_review_summary, restaurant_reviews_overview, and admin_notifications_overview.

API Routes

The current API handler exposes 64 distinct backend route paths through /api/*.

Methods	Route	Audience	Status
ANY	/admin/billing	Admin / Super Admin	Beta / requires integration
ANY	/admin/content	Admin / Super Admin	Working / beta
ANY	/admin/errors	Admin / Super Admin	Working / beta
ANY	/admin/feature-flags	Admin / Super Admin	Working / beta
ANY	/admin/impersonate-partner	Admin / Super Admin	Working / beta
ANY	/admin/integrations	Admin / Super Admin	Beta / requires integration
ANY	/admin/notifications	Admin / Super Admin	Working / beta
ANY	/admin/offers	Admin / Super Admin	Working / beta
ANY	/admin/partners	Admin / Super Admin	Working / beta
ANY	/admin/photo-reward-submissions	Admin / Super Admin	Working / beta
ANY	/admin/reservations	Admin / Super Admin	Working / beta
ANY	/admin/restaurants	Admin / Super Admin	Working / beta
ANY	/admin/reviews	Admin / Super Admin	Working / beta
ANY	/admin/settings/platform-mode	Admin / Super Admin	Working / beta
GET	/admin/stats	Admin / Super Admin	Working / beta
ANY	/ai/actions	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/consumption-uploads	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/consumption/sign-upload	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/demand-forecast	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/events	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/preferences	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
GET	/ai/recommendations	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/recommendations/restaurant	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/restaurant-intelligence	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/route-plan	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/service-time-estimate	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/ai/trends	Guest / Partner / Admin, depending on endpoint	Beta / demo-aware
ANY	/analytics/events	System / Public	Working

Methods	Route	Audience	Status
ANY	/api/index	System / Public	Working
ANY	/auth/forgot-password	Public / authenticated	Working
ANY	/auth/language	Public / authenticated	Working
POST	/auth/login	Public / authenticated	Working
ANY	/auth/logout	Public / authenticated	Working
GET	/auth/me	Public / authenticated	Working
ANY	/auth/reset-password	Public / authenticated	Working
ANY	/auth/security	Public / authenticated	Working
POST	/auth/signup-guest	Public / authenticated	Working
ANY	/auth/verification	Public / authenticated	Working
ANY	/guest/account	Guest	Working
ANY	/guest/favorites	Guest	Working
ANY	/guest/notifications	Guest	Working
ANY	/guest/preferences	Guest	Working
ANY	/guest/privacy	Guest	Working
ANY	/guest/reservations	Guest	Working
GET	/health	System / Public	Working
ANY	/integrations/import-reservations	Partner / Admin	Working
ANY	/partner/integrations	Partner / Admin	Working / beta
ANY	/partner/offers	Partner / Admin	Working / beta
ANY	/partner/photo-reward-submissions	Partner / Admin	Working / beta
ANY	/partner/profile	Partner / Admin	Working / beta
ANY	/partner/reservations	Partner / Admin	Working / beta
GET	/partner/stats	Partner / Admin	Working / beta
ANY	/partner/storage/sign-upload	Partner / Admin	Working / beta
ANY	/privacy/requests	Public / Admin	Working
GET	/public/config	Public	Working
GET	/public/content	Public	Working
POST	/public/follow	Public	Working
GET	/public/offers	Public	Working
GET	/public/restaurants/newest	Public	Working
POST	/public/reviews	Public	Working
GET	/public/rewards/context	Public	Working
POST	/reservations	System / Public	Working
GET	/system/checklists	System / Public	Working

Methods	Route	Audience	Status
GET	/system/feature-status	System / Public	Working

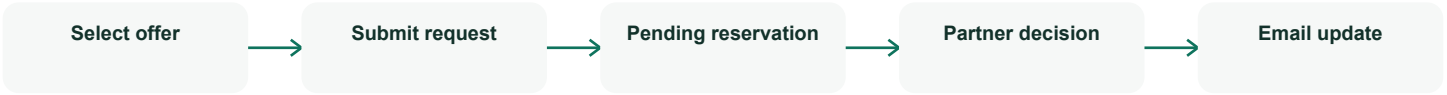
Authentication and Permissions

Supabase Auth is used when configured. Local/demo mode uses seeded in-memory users; passwords are intentionally omitted from this documentation.

Role	Permissions
super_admin	Can manage admin data, switch Platform Mode, control AI demo visibility, and impersonate/view as partners.
admin	Can manage platform data and see current mode but cannot switch Platform Mode.
partner / restaurant	Can manage the linked restaurant, offers, reservations, stats, and feedback only.
guest	Optional account role; anonymous reservation requests are also supported.

Server-side authorization is centralized in requireProfile(headers, roles). Supabase migrations enable RLS policies for scoped table access.

Reservation Flow



- Guest selects a restaurant offer and submits the reservation modal.
- POST /api/reservations validates contact info, offer, party size, date/time, and capacity.
- A pending reservation is created.
- Guest and restaurant notification emails are sent through Resend when configured; otherwise they are logged in demo mode.
- Partner/Admin can accept, reject, cancel, complete, no-show, and add reservation notes.
- Post-visit feedback email/notification support exists for completed reservations.

```
Supported statuses: pending, accepted, rejected, cancelled, completed, requested,
confirmed, no_show
```

Feature Registry

The central feature registry controls visibility across guest, partner, and admin surfaces. The frontend calls canShowFeature(featureKey, options).

English, Spanish, and Hungarian Language Support

The language selector supports English, Spanish, and Hungarian. Language choice is stored in localStorage and can be saved to the authenticated profile through /api/auth/language.

Language	Top-level keys	Literal overrides	Phrase overrides
en	844	0	0
es	844	0	0
hu	1376	970	68

Hungarian has broader literal and phrase overrides to cover older visible text while the app transitions toward key-only translations.

Environment Variables

The following names come from .env.example. No secret values are included.

Variable	Purpose
PORT	Local HTTP server port.
PUBLIC_BASE_URL	Base URL used in links and email templates.
SUPABASE_URL	Supabase project URL.
SUPABASE_ANON_KEY	Public Supabase anon key for server-side Supabase calls.
SUPABASE_SERVICE_ROLE_KEY	Server-side Supabase service role key. Keep secret.
EMAIL_FROM	Verified sender address for transactional email.
RESEND_API_KEY	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
ADMIN_NOTIFICATION_EMAIL	Admin email recipient for notification copies.
SUPABASE_STORAGE_BUCKET	Storage bucket for uploaded media.
VITE_GOOGLE_MAPS_API_KEY	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
IMPERSONATION_SECRET	Server-side secret for Super Admin partner impersonation tokens.
OPENAI_API_KEY	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
VECTOR_DATABASE_URL	Future vector/semantic search database URL.
STRIPE_SECRET_KEY	Future Stripe secret key. Keep secret.
STRIPE_WEBHOOK_SECRET	Future Stripe webhook signing secret. Keep secret.
STRIPE_PRICE_GROWTH_MONTHLY	Future Stripe price identifier.
STRIPE_PRICE_PER_BOOKING	Future Stripe price identifier.
OPENTABLE_CLIENT_ID	Provider integration client identifier.
OPENTABLE_CLIENT_SECRET	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
RESY_CLIENT_ID	Provider integration client identifier.
RESY_CLIENT_SECRET	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
SEVENROOMS_CLIENT_ID	Provider integration client identifier.
SEVENROOMS_CLIENT_SECRET	Provider integration credential. Keep secret unless explicitly public/browser-scoped.

Variable	Purpose
TOCK_CLIENT_ID	Provider integration client identifier.
TOCK_CLIENT_SECRET	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
GOOGLE_RESERVE_CLIENT_ID	Provider integration client identifier.
GOOGLE_RESERVE_CLIENT_SECRET	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
WEATHER_API_KEY	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
LOCAL_EVENTS_API_KEY	Provider integration credential. Keep secret unless explicitly public/browser-scoped.
INTEGRATION_SECRET_ENCRYPTION_KEY	Future encryption key for integration secrets.
RATE_LIMIT_WINDOW_SECONDS	Configuration value documented by .env.example.
RATE_LIMIT_MAX_REQUESTS	Configuration value documented by .env.example.
BACKUP_STORAGE_BUCKET	Configuration value documented by .env.example.

Local Setup

- Use Node 18 or newer.
- Run npm run dev or node server.js from the project root.
- Open http://localhost:4173.
- Without Supabase environment variables, the app runs in local demo mode.
- On this Windows workspace, start-local-server.ps1 starts the local server on port 4173.

npm script	Command
start	node server.js
dev	node server.js
build	npm run check
check	node --check server.js && node --check src/app-core.js && node --check src/push-service.js && node --check api/index.js && node --check public/shared-contracts.js && node --check public/api-client.js && node --check public/shared/layout-shells.js && node --check public/guest/layout.js && node --check public/partner/layout.js && node --check public/admin/layout.js && node --check public/app.js
lint	node scripts/check-static-quality.js
typecheck	npm run check
check:platform-mode	node scripts/check-platform-mode.js
check:signup	node scripts/check-guest-signup.js
check:guest-account	node scripts/check-guest-account.js
check:public-experience	node scripts/check-public-experience.js
check:guest-design-system	node scripts/check-guest-design-system.js
check:route-protection	node scripts/check-route-protection.js
check:routes	node scripts/check-route-map.js
check:architecture	node scripts/check-architecture-foundation.js
test	npm run check:signup && npm run check:guest-account && npm run check:public-experience && npm run check:guest-design-system && npm run check:route-protection && npm run check:routes && npm run check:architecture
docs:pdf	node scripts/generate-docs.mjs

Deployment

- Vercel serves the static frontend and serverless API.
- Supabase migrations provide PostgreSQL schema, views, RLS, storage policies, app settings, and seed/status records.
- Resend is used for live transactional email when configured.
- Service-role keys, email provider keys, impersonation secret, Stripe keys, and integration secrets must remain server-side only.
- vercel.json rewrites /api/:path* to /api/index and SPA routes back to /.

Testing

Command	Purpose
npm run build	Runs the project check command used as the production build gate.
npm run lint	Runs static safety checks.
npm run typecheck	Runs parser/type-shape checks through npm run check.
npm test	Runs signup, guest account, public experience, design system, route protection, route map, and architecture checks.
npm run check	Node syntax checks for server.js, src/app-core.js, api/index.js, public/app.js, shared contracts, layouts, and push service.
npm run check:platform-mode	Validates Platform Mode permissions, persistence, feature visibility, reservation flow, audit/notification logging, and language keys.
npm run check:public-experience	Validates public SEO/mobile/security wiring, public API sanitization, route compatibility, and guest-to-partner reservation flow.
npm run docs:pdf	Regenerates docs/SmartTable-Documentation.md and docs/SmartTable-Documentation.pdf.

There is no full browser screenshot/E2E runner in the current repository; current checks are API/static/route acceptance checks.

Known Issues

- AI modules must stay labeled beta/demo/integration-dependent until backed by live data and measured results.
- OpenTable, Resy, SevenRooms, Tock, Google Reserve, approved reservation APIs, weather, events, Stripe, OpenAI, and vector database integrations are not live.
- Local mode uses in-memory data for most records; only app settings persist to data/app-settings.json.
- Documentation PDF generation requires Python with reportlab, pypdf, and pdfplumber.
- The browser app is concentrated in public/app.js and should eventually be modularized.

Future Roadmap

- Connect approved reservation provider APIs and webhooks.
- Complete restaurant team invitation UI.
- Add background jobs for post-visit email timing, syncs, imports, and AI result attribution.
- Connect Stripe checkout, portal, and webhooks.
- Replace deterministic AI/demo logic with a secure AI service layer.
- Add approved reservation APIs plus separate weather, event, and traffic feeds.
- Add automated browser/E2E tests for guest, partner, admin, and Super Admin workflows.

Scale Architecture Readiness

docs/SmartTable-Scale-Architecture.md documents the scale-readiness audit and safe refactor order.

- Central feature flags separate from Platform Mode.
- Generalized booking metadata with booking_source and booking_status.
- Structured offer-condition fields.
- Review integrity rules.
- Disabled-by-default push architecture.
- SEO, performance, security, and component-library targets.
- Safe implementation order for future growth.